# CE2003 Tut 8

Q1 (a) Referring to the uP16 diagram (without data forwarding), determine the RF contents after the execution of the following instructions:

```
add R2, R3      // R2 <= R2+R3;
add R1, R2      // R1 <= R1+R2;
add R3, R2      // R3 <= R3+R2;
add R4, R2      // R4 <= R4+R2;nop
nop
nop
```

Assume that the RF contents (in Hexadecimal) are: R0-R7 = {0; 1; 2; 3; FFFF; 0; 0; 0}.

(b) Explain why the results are not the expected: R0-R7 = {0; 6; 5; 8; 4; 0; 0; 0}. What needs to be done to the pipeline to fix this problem?

Q2 A simple 5 stage processor is to be implemented in Verilog as per the attached block diagram. The processor has an 18 bit instruction format, while the datapath and data memory are 16 bit. The control unit is implemented as a module in the ID stage (the stage between the IF/ID and the ID/EX pipeline registers). The control unit uses the OpCode (Instruction bits [17:14]) to determine the state of the various multiplexers in the datapath.

Note that the Rd value (Inst[13:11]) accesses Read_data1 which feeds into ALU_Data1_in, and Rs (Inst[10:8]) accesses Read_data2 which feeds into ALU_Data2_in.

Complete the Control module signals (Table 3) for each instruction listed in Table 2. You should refer to the processor block diagram and note that the arithmetic/logical instructions (i.e. **add** to **not**) have the same S1-S3 values (they are essentially the same except for the ALUOp and are all lumped together as a single operation and are referred to as ROP in table 3).

You need to determine the state of the control signal for the 3 multiplexers (S1-S3), below, as well as the ALU control signal (EX_ALUOp), and the memory and register file control signals.

S1: sel_PC
S2: sel_ALUSrc1
S3: sel_Mem2Reg

Q3 You are required to modify the processor pipeline to include a jump instruction.

Modify the processor pipeline to allow for the *jump and link register* instruction. This instruction takes the contents of a register file location and assigns it to the PC, and at the same time stores the current PC into the register file.

Note: if rd = 0, the PC is not saved. Also note that this an R-format instruction with Opcode D, not a J-format instruction.

| | | | | |
|---|---|---|---|---|
| **jlr** | 0x0D | 0x00 | **jlr** *rd*, *rs* | R[rd]= (pc+1); pc = R[rs]; |

**Table 1: The 18 bit instruction formats for the uP16 processor**

| 17 — 14 | 13 — 11 | 10 — 8 | 7 — 0 |
|---|---|---|---|
| **Field** | **4 bits** | **3 bits** | **3 bits** | **8 bits** |
| R-format | OpCode | Rd | Rs | Function/Immed |
| J-format | OpCode | Target address (14 bits) | | |

Note: J-format instructions are not implemented in the processor block diagram

**Table 2: The instruction set for the uP16 processor**

| Name | op | Funct/ Immed | Instruction | Effect(s) |
|---|---|---|---|---|
| **nop** | 0x00 | 0x00 | **nop** | Do nothing (instruction word = 0) |
| **add** | 0x00 | 0x01 | **add** *rd, rs* | RF[rd] = RF[rd] + RF[rs]; |
| **sub** | 0x00 | 0x02 | **sub** *rd, rs* | RF[rd] = RF[rd] - RF[rs]; |
| **and** | 0x00 | 0x03 | **and** *rd, rs* | RF[rd] = RF[rd] **and** RF[rs]; |
| **or** | 0x00 | 0x04 | **or** *rd, rs* | RF[rd] = RF[rd] **or** RF[rs]; |
| **nand** | 0x00 | 0x05 | **nand** *rd, rs* | RF[rd] = RF[rd] **nand** RF[rs]; |
| **nor** | 0x00 | 0x06 | **nor** *rd, rs* | RF[rd] = RF[rd] **nor** RF[rs]; |
| **xor** | 0x00 | 0x07 | **xor** *rd, rs* | RF[rd] = RF[rd] **xor** RF[rs]; |
| **not** | 0x00 | 0x08 | **not** *rd, rs* | RF[rd] = **not** RF[rs]; |
| | | | | |
| **mov** | 0x00 | 0x0A | **mov** *rd, rs* | RF[rd] = RF[rs]; |
| | | | | |
| **lw** | 0x01 | immed | **lw** *rd, rs, immed* | RF[rd] <- memory(RF[rs]+immed); |
| **sw** | 0x02 | immed | **sw** *rd, rs, immed* | RF[rd] -> memory(RF[rs]+immed); |
| | | | | |
| **lli** | 0x03 | immed | **lli** *rd, rs, immed* | RF[rd] = sign_ext(immed); (*rs* is unused) |
| **lui** | 0x04 | immed | **lui** *rd, rs, immed* | RF[rd] = {immed[7:0], RF[rs][7:0]} |
| **addi** | 0x05 | immed | **addi** *rd, rs, immed* | RF[rd] = RF[rs] + sign_ext(immed); |
| | | | | |
| | 0x6-7 | | | **Reserved** |
| | | | | |
| **beq** | 0x08 | immed | **beq** *rd, rs, immed* | if (RF[rd] = = RF[rs]) pc += sign_ext(immed) + 1; |
| **bne** | 0x09 | immed | **bne** *rd, rs, immed* | if (RF[rd] != RF[rs]) pc += sign_ext(immed) + 1; |
| **blt** | 0x0A | immed | **bgtz** *rd, rs, immed* | if (RF[rd] < RF[rs]) pc += sign_ext(immed) + 1; |
| **bgt** | 0x0B | immed | **bgtz** *rd, rs, immed* | if (RF[rd] > RF[rs]) pc += sign_ext(immed) + 1; |
| | | | | |
| | 0xC-F | | | **Reserved** |

**Table 3: Control Signals**

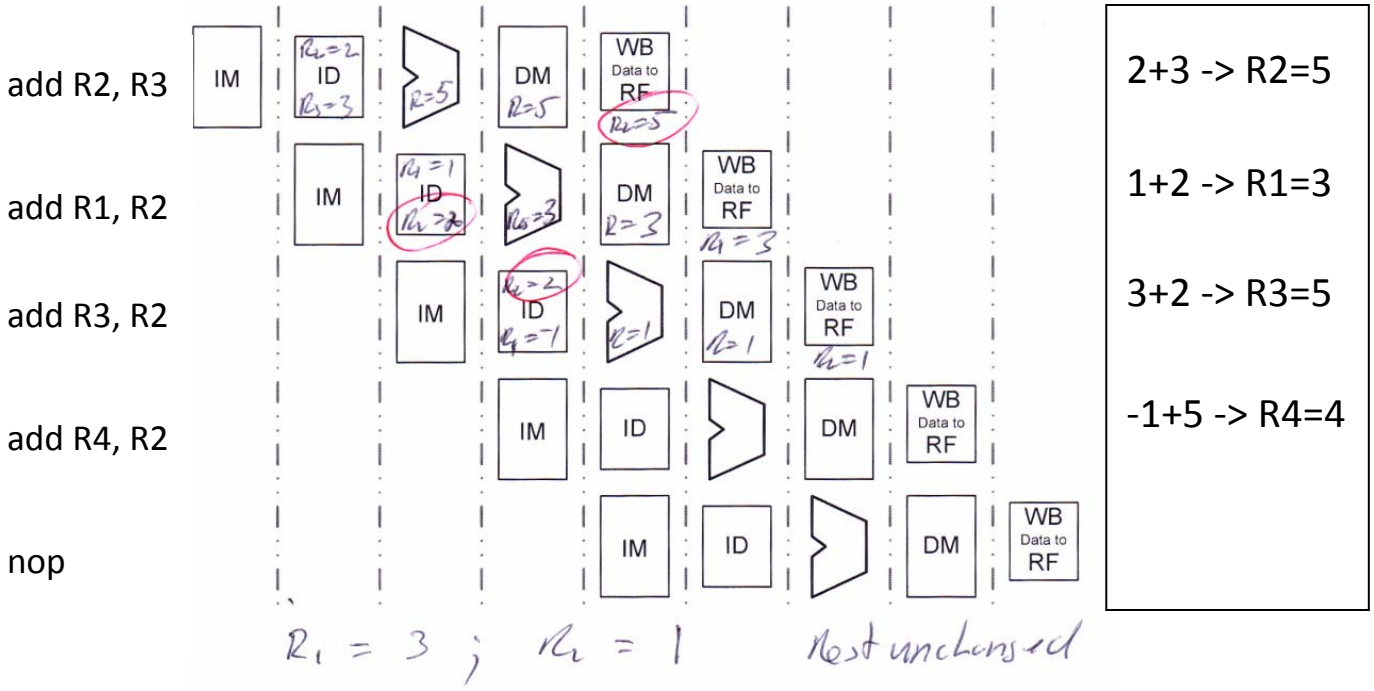
| Inst | S1 | S2 | S3 | RF Write | Mem Write | Mem Enable | ALUOp |
|---|---|---|---|---|---|---|---|
| **NOP**<br>**ROP**<br>**MOV**<br>**LW**<br>**SW**<br>**LLI**<br>**LUI**<br>**ADDI**<br>**BR**<br>**Default** | **0** | **X** | **X** | 0 | 0 | 0 | 0000 |

## Answers

Q1 Determine the RF contents after the execution of:

```
add R2, R3      // R2 <= R2+R3;
add R1, R2      // R1 <= R1+R2;
add R3, R2      // R3 <= R3+R2;
add R4, R2      // R4 <= R4+R2;
nop
nop
```

Assume that the RF contents (in Hex) are: R0-R7 = {0; 1; 2; 3; FFFF; 0; 0; 0}.




Result is: R0-R7 = {0; **3**; **5**; **5**; **4**; 0; 0; 0}.

The expected result is R0-R7 = {0; **6**; **5**; **8**; **4**; 0; 0; 0}.

First R2=2+3=5; Then R1=1+5 = 6; Then R3=3+5=8; Then R4=-1+5=4

Q1b The results are not as expected due to the use of register R2 in the 2nd and 3rd instructions.

The register file is only updated 5 cycles after the start of the instruction, so the 2nd and 3rd instructions use the original R2 value (i.e. R2 =2) and not the value calculated in the 1st instruction. To solve this problem, data forwarding would need to be implemented. This could be used to pass the result stored in the pipeline registers just after the ALU (and just after the data memory stage) directly into the ALU inputs.

Q2

Most of the ALUOp values are fairly obvious. For ROP and MOV, they are just the function values (let the ALU work out what needs to be done later). For LW and SW, they are an add, as the ALU needs to add the RF contents with the immediate value to get the memory address. Eg. memory(RF[rs]+immed)

For LLI, the ALU performs a similar action as for the MOV instruction (except it is on the other ALU input), so assign it an unused ALUOp code and let the ALU work it out (0x0B). For LUI, the ALU does something strange (ALUResult = {ALUData2[7:0], ALUData1[7:0]};), so lets give it its own ALUOp code (0x0C).

For ADDI, the ALU performs an add (0x01). Eg. RF[rs] + sign_ext(immed)

For the BR instructions, we do not use the ALU so set it to the default (whatever that is).

For the muxes:

**S1 (sel_PC)** this is used for all instances where we get a change in the PC (BR inst.)

**S2 (sel_ALUSrc1 )** can be the RF *Read Data 1* (i.e. 0) except for LW, SW, LLI, LUI, ADDI.

**S3 (sel_Mem2Reg)** only used for a LW instruction. All don't cares can default to 0 as they do not do a register file writeback.

| Inst | S1 | S2 | S3 | RF Write | Mem Write | Mem Enable | ALUOp |
|---|---|---|---|---|---|---|---|
| **NOP** | **0** | ~~**X**~~ **0** | ~~**X**~~ **0** | ~~X~~ 0 | 0 | 0 | 0000 |
| **ROP** | 0 | 0 | 0 | 1 | 0 | 0 | Inst[3:0] |
| **MOV** | 0 | ~~**X**~~ **0** | 0 | 1 | 0 | 0 | Inst[3:0] |
| **LW** | 0 | 1 | 1 | 1 | 0 | 1 | 0001 |
| **SW** | 0 | 1 | X | 0 | 1 | 1 | 0001 |
| **LLI** | 0 | 1 | 0 | 1 | 0 | 0 | 1011 |
| **LUI** | 0 | 1 | 0 | 1 | 0 | 0 | 1100 |
| **ADDI** | 0 | 1 | 0 | 1 | 0 | 0 | 0001 |
| **BR** | 1? | ~~X~~ 0 | ~~X~~ 0 | 0 | 0 | 0 | ~~XXXX~~ 0000 |
| **Default** | **0** | **0** | **0** | **0** | **0** | **0** | **0000** |

Set RFwriteEnab default to 0 as we do not really want to do a write if we accidently get a default situation

**Next 2 pages are just for explanation.**

```
module Control #(parameter ISIZE=18, DSIZE=16) (
    input [ISIZE-1:0] inst,
    input [DSIZE-1:0] readData1, readData2,
    output reg PC_select , sel_ALUsrc1, mem2Reg,
    output reg RFwriteEnab, memWriteEnab, memEnab,
    output reg [3:0] ALUop );

    // define op codes
    parameter OP0=0, LW=1, SW=2, LLI=3, LUI=4, ADDI=5, BEQ=8, BNE=9, BLT=10, BGT=11;

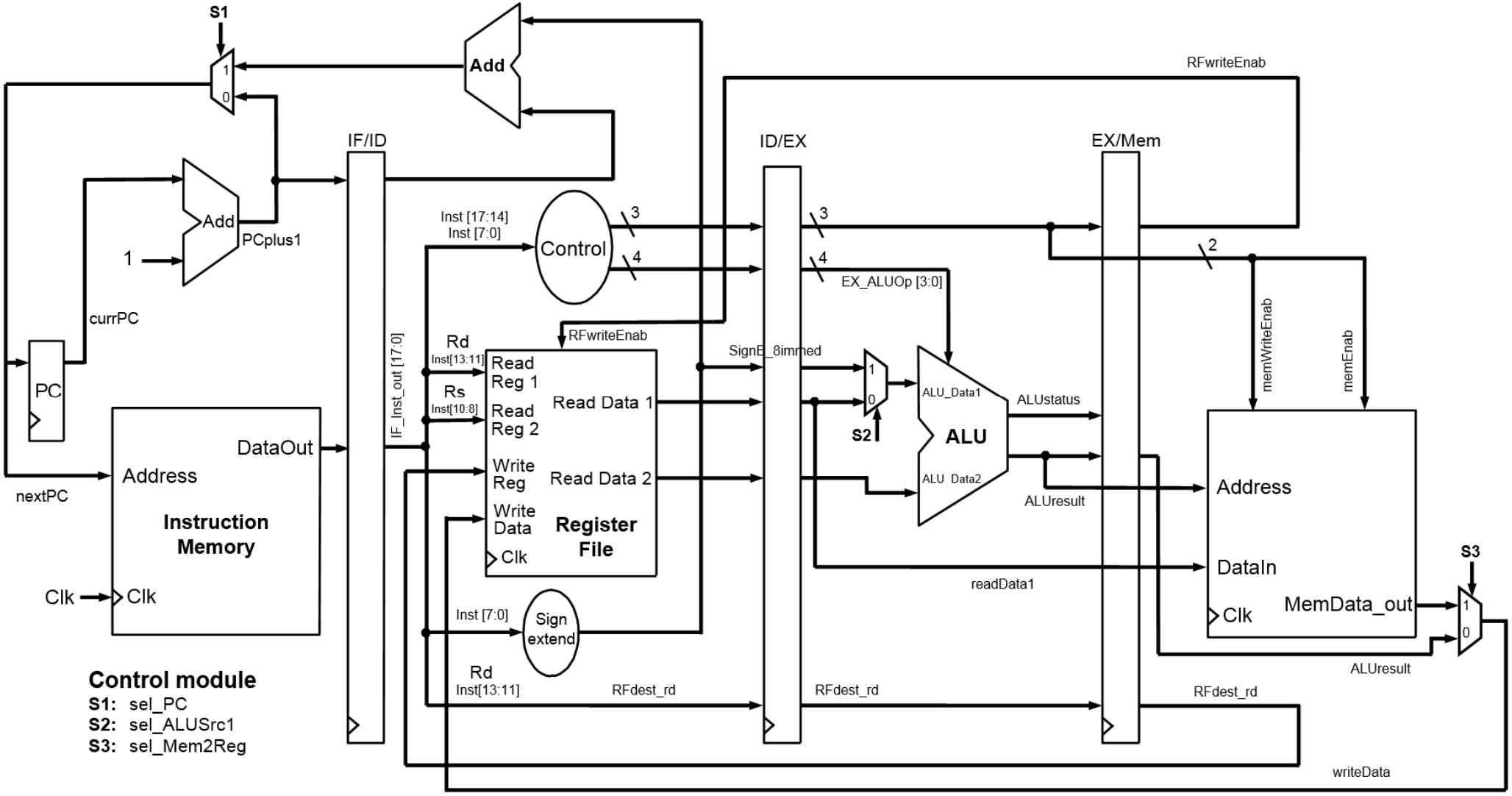
    always @ * begin
        ALUop = 4'b0000;              // initialise default values // (NOP)
        PC_select = 1'b0;             // S1: def = 0  // def is PC=PC+1
        sel_ALUsrc1 = 1'b0;           // S2: def = 0  // (RF_Data 1)
        mem2Reg = 1'b0;               // S3: def = 0  // default is ALU result writeback
        RFwriteEnab = 1'b0;           // def = 0      // default is No RF write
        memWriteEnab = 1'b0;          // def = 0      // default is No memory write
        memEnab = 1'b0;               // def = 0      // default is memory disabled

        case(inst[17:14])             // The OPCode
            OP0 :   begin
                ALUop = inst[3:0];    // ALUOp is bottom 4 bits of Function field
                RFwriteEnab = 1'b1;   // write to RF
            end

            LW  :   begin
                ALUop = 4'b0001;      // ALU does an ADD (Rs+immed)
                sel_ALUsrc1 = 1'b1;   // S2: Immed on data1
                mem2Reg = 1'b1;       // S3:
                RFwriteEnab = 1'b1;   // write to RF
                memEnab = 1'b1;
            end
                                      .
                                      .
                                      .
            BEQ :   begin
                PC_select = (readData1 == readData2) ? 1'b1 : 1'b0;     // S1:
            end
                                      .
                                      .
                                      .
        endcase
    end

endmodule       // End of Control module
```

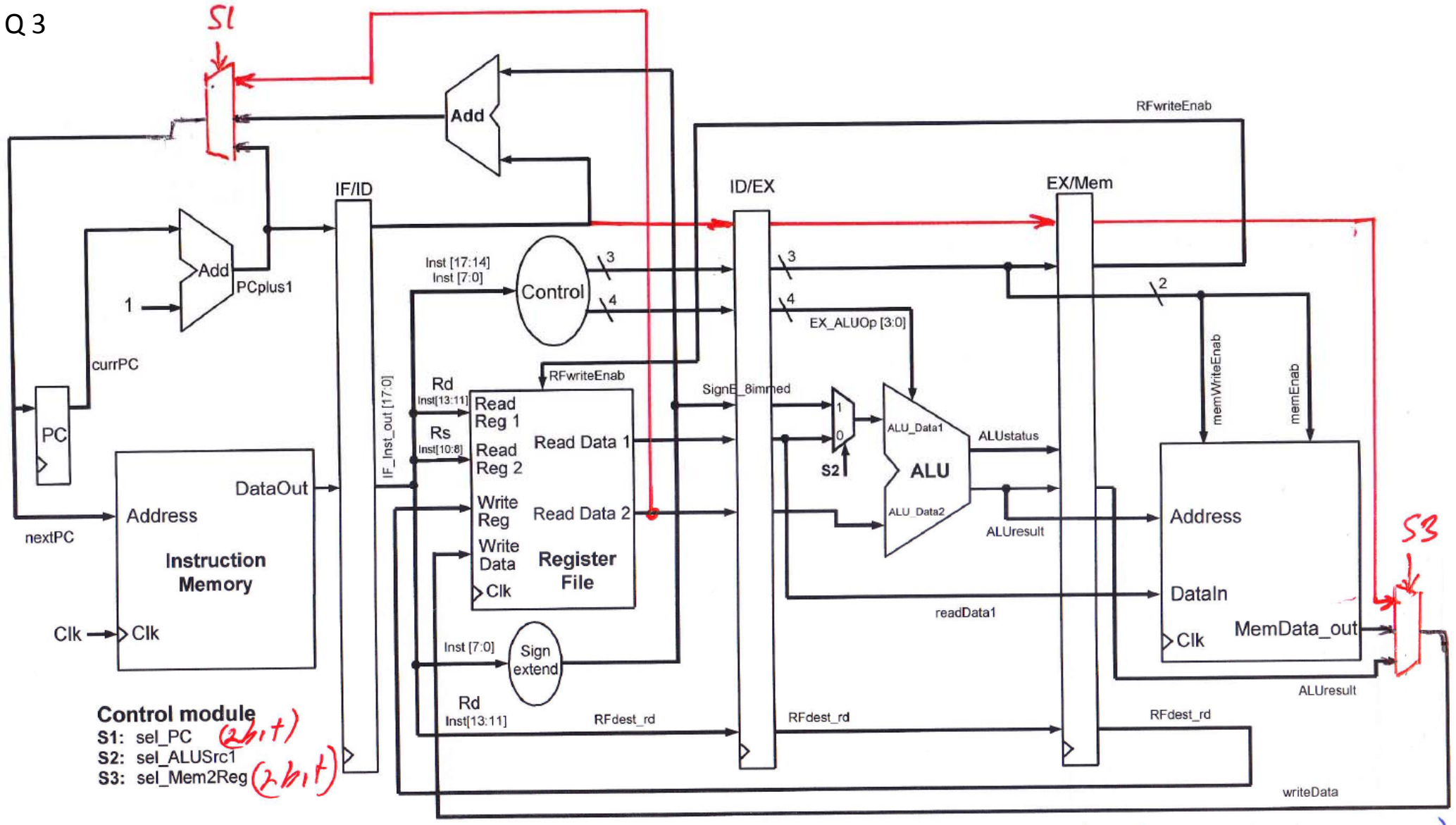
```
case (inst[17:14])          // The OPCode
    OP0 :    begin                  // Covers all opcode = 0000 instructions
        ALUop = inst[3:0];          // ALUOp is bottom 4 bits of Function field
        RFwriteEnab = 1'b1;         // write to RF
    end
    LW      :    begin
        ALUop = 4'b0001;            // ALU does an ADD (Rs+immed)
        sel_ALUsrc1 = 1'b1;         // S2: Immed on data1
        mem2Reg = 1'b1;             // S3:
        RFwriteEnab = 1'b1;         // write to RF
        memEnab = 1'b1;
    end
    SW  :    begin
        ALUop = 4'b0001;            // ALU does an ADD (Rs+immed)
        sel_ALUsrc1 = 1'b1;         // S2: Immed on data1
        memWriteEnab = 1'b1;
        memEnab = 1'b1;
    end
    LLI :    begin
        ALUop = 4'b1011;            // ALU does flow through on Rd (similar to mov (0x0A))
        sel_ALUsrc1 = 1'b1;         // S2: Immed on data1
        RFwriteEnab = 1'b1;         // write to RF
    end
    LUI :    begin
        ALUop = 4'b1100;            // ALU: ALUResult = {ALUData1[7:0], ALUData2[7:0]};
        sel_ALUsrc1 = 1'b1;         // S2: Immed on data1
        RFwriteEnab = 1'b1;         // write to RF
    end
    ADDI :   begin
        ALUop = 4'b0001;            // ALU does an ADD (Rd <- Rs+immed)
        sel_ALUsrc1 = 1'b1;         // S2: Immed on data1
        RFwriteEnab = 1'b1;         // write to RF
    end
    BEQ  :   begin
        PC_select = (readData1 == readData2) ? 1'b1 : 1'b0;      // S1:
    end
    BNE :    begin
        PC_select = (readData1 != readData2) ? 1'b1 : 1'b0;      // S1:
    end
    BLT :    begin
        PC_select = (readData1 < readData2) ? 1'b1 : 1'b0;       // S1:
    end
    BGT :    begin
        PC_select = (readData1 > readData2) ? 1'b1 : 1'b0;       // S1:
    end
    default: begin
        ALUop = 4'b0000;            // default not really needed, Covered above
    end
endcase
```




**Control module**

**S1:** sel_PC
**S2:** sel_ALUSrc1
**S3:** sel_Mem2Reg

Q 3




i) S1 needs 2bits to select between the 3inputs (PCplus1; BR; JMP)

ii) PCplus1 needs to be fed through the pipeline. S3 needs to be 3 bits to select between the 3 possible inputs.

Q1 (sup) Assume that the RF contents are: R0-R7 = {0; 1; 2; 3; FFFF; 0; 0; 0}.

